Final Report
Anvil Load Testing Framework

Shubham Prashar • SP4540

Contents

---

1. Synopsis
2. System Architecture
3. Comparison with Existing Tools
4. Research Questions
5. Methodology & Evaluation Metrics
6. Discussion & Limitations
7. LLM Design and Integration
8. Future Work
9. Self-Evaluation
10. Deliverables

Synopsis

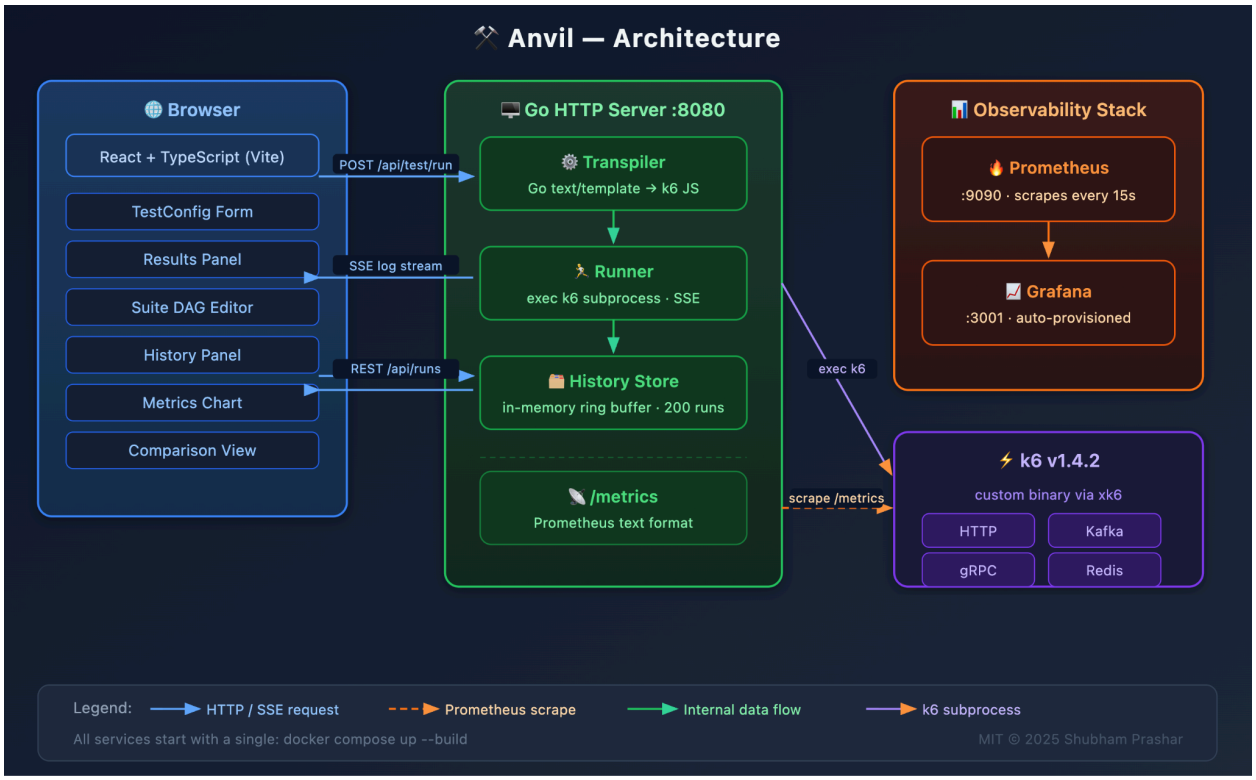
Anvil is an open-source, UI-driven load testing framework that requires no coding or scripting. It is driven by a simple yet powerful UI. There are existing powerful solutions like k6, Gatling, and Locust, but they require the user to write and maintain multiple scripts.

Other no-script solutions like BlazeMeter are expensive, and the org needs to pay for the license every month, which adds up fast. Then there is JMeter, which sits at the intersection of these two; it is both open-source and has a UI, but it is not browser-based, there is no real-time metric streaming, and it relies on XML configurations that become difficult to manage when scaling increases.

Anvil is written over a popular open-source load testing framework, k6. I used React and TypeScript to create a single-page application that acts as a real-time streaming layer for showing the metrics. An LLM layer that helps junior developers to make more sense of the raw numbers. Multiple protocols are being supported out of the box, like HTTP, gRPC, Kafka, and Redis. The whole framework spins up with a single `docker-compose up –build` command.

It might look like Anvil is not solving anything, but Anvil is an intersection of all these tools and open source which makes it free for all. A UI-driven load testing framework that makes sure that the developer is not getting overloaded with writing scripts and maintaining them. A dashboard where they can monitor real-time metrics. LLM capabilities, which will grow over time. Open-sourced and freely available on GitHub. In my personal experience, every startup I worked with either wrote their own wrapper over k6 or simply skipped load testing. Anvil can act as one framework used by all the startups trying to escape the expensive licensing.

System Architecture



There are multiple fields in the UI that prompt the user to enter their config; this is then sent to the backend server written in Go, which transforms it into a config that k6 can understand and starts a k6 subprocess, and the results are streamed on the frontend as the test continues.

Prometheus scraps the metrics that can be plotted using Grafana, more metrics can be emitted as the project grows. The intent behind this simple architecture was to make sure there is no friction for open-source contributions and it is kept as simple as the UI. To further reduce the complexity, everything runs from a single docker-compose file, which ensures that the user is not confused when running or deploying and everything is managed from a single point.

The LLM layer can be grown further, current setup generates a prompt that we discover from the load testing and sends it to the selected LLM. My next objective in the summer break is to spin up a local LLM model, experiment and see how I can further improve the LLM use case in the load testing workflow.

Evaluation against existing Tools

Tool	Requires Custom Scripts	Web UI	Multi-Protocol	Real-Time Streaming	LLM Integration	License
k6	Yes (JavaScript)	No (CLI only)	HTTP, gRPC, Kafka*	No	No	Free / OSS
Locust	Yes (Python)	Basic (read-only)	HTTP only	No	No	Free / OSS
JMeter	XML config (complex)	Desktop GUI	HTTP, FTP, JDBC	No	No	Free / OSS
Gatling	Yes (Scala DSL)	No (CLI only)	HTTP, WebSocket	No	No	Free / OSS
Artillery	YAML + JS hooks	No (CLI only)	HTTP, WebSocket	No	No	Free / OSS
BlazeMeter	No (GUI)	Yes (full)	HTTP, gRPC	Yes	No	Commercial ($500+/mo)
Anvil	No (GUI)	Yes (full)	HTTP, gRPC, Kafka, Redis	Yes (SSE)	Yes (built-in)	Free / OSS

Why was k6 chosen as the underlying engine for Anvil?
Some pointers considered before selecting k6 were

- The metrics are well structured in the JSON format, which makes it pretty easy to ingest by the backend server and a common format for data exchange between the frontend and backend.
- We don't need to parse the logs to understand if the load testing is failing or if there is some unexpected issue, as the signals by k6 are pretty easy to interpret.
- Already well known and widely used across various startups, so the developers already have an idea when debugging in case something goes wrong in their self hosted Anvl.
- Well maintained and open-sourced
- Other frameworks were either paid, missing protocols, or just not well supported by the community

Research Questions

- **Can we reduce the friction of load testing by using a UI-based framework?**
  Yes. The friction with other frameworks is that it requires the user to write scripts or configs, and then the user has to maintain them. Anvil is a no-code framework, meaning the user just needs to know the obvious details about their resource, and they can start testing.

- **Can a directed acyclic graph be used to capture and replicate how real-world resources are executed in a sequence?**
  Not a strong yes. This is because DAGs are directed acyclic graphs, and there are no loops (node 1 is not executed after node 2). The problem is that this might not represent the real-world scenario. But DAG-based testing can be modified/extended in the later stages.

- **How can LLM be used to make load testing results easier to interpret?**
  LLM can be used to understand the results much better and see if there are some outlier in the metrics. This is more powerful when the user is someone like a junior engineer who might not even know what a good metric is or where to raise the alarm and optimize.

- **Can LLM be used to fully automate load testing?**
  We need to wait and see how Anvil evolves. We need to be careful about load testing as it eats the resource fast and is meant to bombard a service just to test its limits. That being said, in near future, LLMs can definitely be used to automate the load testing and be added to the deployment pipeline and they may only raise alerting if there is a genuine deviation in the metrics.

Methodology & Evaluation Metrics

To test Anvil, resources available over the internet were used, such as publicly available endpoints. I was able to observe how, when bombarded with requests, public endpoints like Google start to rate limits, and Anvil was able to identify these failing requests. There are no standard metrics or benchmarking framework to evaluate Anvil, but the sanity was done over these publicly available free resources.

A comprehensive test suite is underway and will be committed and pushed in the next stage of the project along with suggestions for the Grafana dashboard.

Discussion & Limitations

Anvil is as easy as it gets to load test services, and it completely removes the friction and overhead of writing and maintaining scripts while also being open source and free to use. I feel the evaluation was quite limited, as, like any open-source project, things change with time, and the best evaluation would be how this project gains traction in the open-source community.

Although DAG tries to mimic most cases of multiple resources being used in a certain order, real world softwares/resources may not follow this ordering and Anvil fails to mimic them. That being said, this can be worked on.

LLM Design and Integration

The user is not expected to export any data from Anvil and manually ask LLM. Rather, they need to configure Anvil for the LLM of their choice, and it will generate the prompt and ask the chosen LLM to summarize the results. These features can further be improved and worked on to make sure even the configuration is easy to do. The result is a very easy-to-understand summary in English that helps the user know if they need to further dig in the code to debug this or if everything looks alright.

Future Work

- Currently LLM is used mainly for post-run summaries, there needs to be further research into how we can use LLM to make the load testing more robust.
- Natural language to load-testing needs to be researched upon and integrated if there is a demand in the industry.
- A persistent storage of data needs to be added.
- As the project is adapted by multiple organisations, there needs to be a fine tuning of the resources being used/blocked by Anvil to make sure the cost is perfectly optimised.
- An extension of DAG needs to be worked on to make sure it covers the cases which are not directed and acrylic.

What did I learn?

High-scale startups in India prefer writing their own load testing framework (which is much easier due to LLMs) rather than licensing something like BlazeMeter. Since the framework is not standardized, there are often gaps in them, and latency-causing issues are often missed and shipped to production.

I also learned that some companies even skip load testing because it is too much work or the friction is very high for the engineers and causes delays. Engineering is not a problem here, but rather addressing the small issues and having a single repo where every company can commit and push their features for load testing is needed.

What Was accomplished?

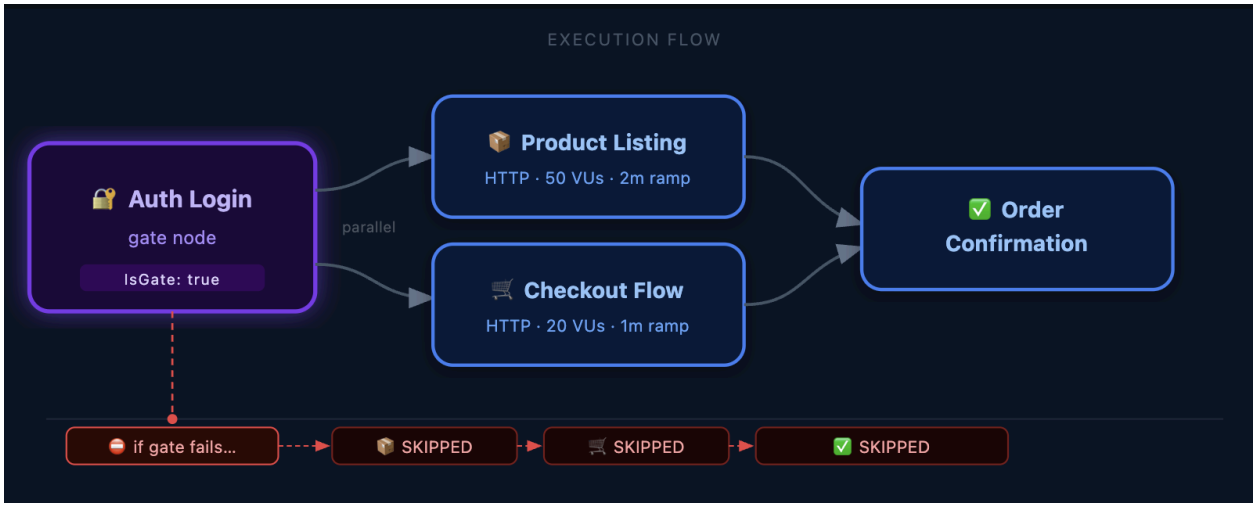
- UI-based load testing for a single resource.
- DAG based UI for load testing resources in a particular order (with dependencies).
- Golang backend that uses k6 subprocesses for handling load testing.
- React and TypeScript-based frontend to render all the data.
- A dashboard where real-time metrics are streamed with live logs of k6.
- A config generator that takes the data from the UI and converts it into a config that k6 understands.
- Support for HTTP, gRPC, Kafka, and Redis.
- History and comparison (this needs to be made persistent).
- LLM for explaining the results (OpenAI, Anthropic, Ollama, and a local mock setup).
- Easy to run with a single docker compose command.
- Prometheus + Grafana for monitoring Anvil itself.

What was most challenging?
Being a backend engineer, the frontend was quite challenging to think through, especially to make sure the user requires no coding to test. The DAG UI was something I especially enjoyed and was challenging.

Where was AI used?
Claude was used to help me debug and write complex code. The documentation was also prettified and paraphrased to make sure it is easier and more comprehensive for the open-source community. Diagrams were generated with the help of AI, which converted basic diagrams into something pretty and easier to understand for the readers..

What is a dag?

According to [wikipedia]
"In mathematics, particularly graph theory, and computer science, a directed acyclic graph (DAG) is a directed graph with no directed cycles. That is, it consists of vertices and edges (also called arcs), with each edge directed from one vertex to another, such that following those directions will never form a closed loop. A directed graph is a DAG if and only if it can be topologically ordered, by arranging the vertices as a linear ordering that is consistent with all edge directions. DAGs have numerous scientific and computational applications, ranging from biology (evolution, family trees, epidemiology) to information science (citation networks) to computation (scheduling)."



Deliverables

Code and documentation is available on this public repository over github

https://github.com/oyeprashar/anvil-load-tester